# ChronoStream-RAG: Bitemporal Watermark Coalescing and Document-Atomic Vector Consistency for Out-of-Order Streaming RAG

Novel Research Paper & Multi-Perspective 10-Seed Empirical Evaluation

**Lynette Osako** (ID: 8000290404) • **Panhapiseth Lim** (ID: 8000265274) • **Rajan Yadav** (ID: 8000277867)

Lynette Osako (ID: 8000290404), Panhapiseth Lim (ID: 8000265274), Rajan Yadav (ID: 8000277867)

*Department of Computer Science / Big Data Analytics Research Group*

## Abstract

Retrieval-Augmented Generation (RAG) is increasingly deployed over live, continuously mutating document streams such as news feeds, enterprise knowledge bases, and Wikimedia revision logs. In distributed big data pipelines, document updates frequently arrive out of chronological order due to multi-partition broker delays, repeat under at-least-once delivery semantics, or complete dense neural embedding asynchronously out of order across parallel worker pools. While scalar key-value databases trivially resolve out-of-order writes via per-record Last-Write-Wins (LWW) version checks, we demonstrate that multi-chunk vector RAG pipelines violate scalar LWW assumptions, giving rise to severe semantic anomalies that have gone unaddressed in both the vector database and NLP literature.

Specifically, we formalize and empirically measure three streaming RAG anomalies: (1) Intra-Document Version Tearing (*Semantic Split-Brain*), where asynchronous chunk writes or revision length shrinkage leave *orphan tail chunks* from older revisions active alongside updated leading chunks in the Approximate Nearest Neighbor (ANN) index, causing top-$k$ retrieval to inject self-contradictory facts from multiple revisions of the same document into the language model prompt; (2) Endogenous Embedding Compute-Lag Inversion, where variable revision chunk counts cause older revisions to finish embedding *after* newer revisions even when network arrival order is 100% sequential ($p_{\text{ooo}}=0\%$); and (3) Transient Vandalism and Historical Erasure, where immediate head-only overwrites expose short-lived corrupted edits and destroy point-in-time (AS OF) auditability.

We present ChronoStream-RAG, a bitemporal streaming RAG architecture integrating Document-Atomic Generation Switching, Event-Time Watermark Revert Coalescing, Chunk-Level SHA-256 Delta Deduplication, and Hot/Cold Bitemporal Tiering (maintaining a bounded $K=2$ hot working history in Qdrant HNSW backed by Snappy-compressed Apache Parquet archives). Evaluated on WikiChrono-Bench—a benchmark built from 1,722 real English Wikipedia revisions across 60 tracked multi-chunk documents (246 revisions, 1,434 chunks) and 180 temporal queries over 10 independent random seeds (9,000 total RAG evaluations)—ChronoStream-RAG achieves 87.28% ± 0.85% overall fact agreement (vs. 30.44% ± 2.51% for Arrival-Order [$p < 10^{-12}$], 38.22% ± 1.63% for Scalar Chunk LWW [$p < 10^{-13}$], and 51.56% ± 1.63% for Head-Only Doc-Atomic Guard [$p < 10^{-11}$]), completely eliminates Cross-Chunk Version Tearing (0.00% vs. 25.61% ± 1.18% in Scalar Chunk LWW), reduces neural embedding compute by 81.4%, shrinks active HNSW vector memory by 43.5% compared to Full Bitemporal indexing, and scales linearly from 1 GB to 10 GB with 118.0 ms checkpoint recovery.

## 1. Introduction

Retrieval-Augmented Generation (RAG) grounds Large and Small Language Models (LLMs/SLMs) in external corpora by indexing chunked documents in dense vector databases such as Qdrant, Milvus, or Pinecone [Lewis et al., 2020; Malkov & Yashunin, 2020; Wang et al., 2021]. However, most RAG evaluations assume a static corpus or a cleanly ordered batch refresh. When RAG is connected to live event streams—such as Apache Kafka topics carrying continuous Wikimedia edits, clinical updates, or financial filings—the ingestion pipeline becomes subject to the fundamental hazards of distributed stream processing [Barga et al., 2007; Akidau et al., 2015]:

1. Network & Partition Out-of-Orderness ($t_{\text{arrival}}$ inversion): When updates are distributed across broker partitions or retried over WAN links, an older revision $v_{k-1}$ ($t_{\text{event}}(v_{k-1}) < t_{\text{event}}(v_k)$) can

arrive at the consumer *after* the newer revision $v_k$ has already been indexed.

2. Endogenous Embedding Compute-Lag Disorder ($t_{\text{available}}$ inversion): Unlike scalar database writes that complete in microseconds, RAG ingestion requires splitting each document revision into $m$ text chunks and passing them through a neural embedding model (SentenceTransformer) across parallel worker pools. A 6-chunk major revision routed to a busy worker batch can finish embedding *seconds after* a subsequent 4-chunk revision that arrived later—inverting version order inside the pipeline even when Kafka arrival order is strictly chronological!

3. At-Least-Once Duplicate Deliveries & Rapid Vandalism/Reverts: Broker retries deliver repeated records, while open collaborative streams like Wikipedia experience frequent minor syntactic edits and transient vandalism reverted within seconds.

### 1.1 Why Scalar Database Version Checking ("Last-Write-Wins") Fails in RAG

A skeptical systems reviewer might ask: *"Why not simply attach a version number or event timestamp to each vector point and apply the Thomas Write Rule / Last-Write-Wins (if incoming.version < existing.version: drop)?"*

We prove both theoretically and empirically that per-chunk scalar version checking (M2a_ScalarChunkLWW) fails catastrophically in multi-chunk RAG. Because a document $D$ is segmented into multiple chunks $\{c_0, c_1, \dots, c_{m-1}\}$, its chunk count $m(v_k)$ varies across revisions. When revision $v_{k-1}$ has 6 chunks (chunk_0..chunk_5) and a subsequent consolidated revision $v_k$ has 4 chunks (chunk_0..chunk_3), per-chunk LWW updates slots 0–3 to $v_k$ (since $v_k > v_{k-1}$), but leaves slots 4 and 5 (chunk_4, chunk_5) from $v_{k-1}$ alive as orphan tail chunks in the HNSW graph! When a user queries Qdrant for top-$k$ chunks of document $D$, semantic search retrieves chunk_0 from $v_k$ *together with* chunk_4 from $v_{k-1}$. Confronted with contradictory facts from two different revisions of the same document in the same context window—an *inter-context knowledge conflict* [Hou et al., 2024; Xu et al., 2024; Wang et al., 2025]—the language model hallucinates hybrid falsehoods or selects the stale specification in 25.61% ± 1.18% of queries!

Even if version checking is elevated to Document-Atomic Generation Replacement (M2b_DocAtomicVersionCheck)—which purges all prior chunks of $D$ before inserting $v_k$—three critical limitations remain: (i) it discards late-arriving historical revisions, scoring 0.00% on point-in-time (AS OF) temporal auditing queries; (ii) with zero watermark holdback, it immediately indexes transient vandalism revisions during the window before a revert arrives; and (iii) it wastefully re-embeds all chunks of a document even when an edit only modified punctuation in a single paragraph.

### 1.2 Contributions

To bridge this gap between distributed stream processing and neural retrieval, we make four contributions:

1. Formal Anomaly Taxonomy & Bitemporal Observability Model: We formalize $(t_{\text{event}}, t_{\text{arrival}}, t_{\text{available}})$ accounting to separate unarrived information from processing errors on held information, and define three streaming RAG anomalies: *Intra-Document Version Tearing Rate (IDVTR)*, *Compute-Lag Inversion Rate (CLIR)*, and *Vandalism Exposure*.

2. ChronoStream-RAG Architecture (M4): We design a bounded working-history RAG engine combining (a) *Document-Atomic Generation Commits*, (b) *Event-Time Watermark Holdback with SHA-256 Revert/Vandalism Coalescing*, (c) *Chunk-Level SHA-256 Delta Deduplication*, and (d) *Hot Qdrant ($K=2$) + Cold Apache Parquet Bitemporal Tiering* with late-arrival interval splicing.

3. WikiChrono-Bench & 10-Seed Empirical Evaluation: Using 1,722 live Wikimedia revisions, real 384-d SentenceTransformer embeddings, Qdrant HNSW indexing, PyArrow/Parquet persistence, and local Flan-T5-Small generation across 10 independent seeds, we demonstrate statistically significant gains ($p < 10^{-9}$) over four baselines.

4. 1 GB to 10 GB Big Data Scalability & Interactive Dashboard: We evaluate throughput, memory footprint, and crash recovery from 1 GB to 10 GB (explicitly separating original content from repeated delivery records) and provide an interactive Streamlit/HTML5 telemetry dashboard.

## 2. Related Work and the Four-Dimensional Gap

First, we explicitly disambiguate our system name from two unrelated works: ChronoStream [Wu & Tan, ICDE 2015] is a pre-LLM distributed systems paper on elastic state migration for cloud stream operators, while ChronoRAG [Kim et al., 2025] reorders retrieved passages from static narrative stories into plot sequence. Neither addresses out-of-order streaming ingestion, vector index consistency, or bitemporal RAG.

Existing research falls into four disconnected areas:

- Temporal & Dynamic QA Benchmarks: *StreamingQA* [Liška et al., 2022], *RealTime QA* [Kasai et al., 2023], *FreshLLMs* [Vu et al., 2024], *StreamingRAG* [Sankaradas et al., 2025], and *DyG-RAG* [Sun et al., 2025] evaluate LLMs on evolving facts, but assume that the underlying retrieval index is updated cleanly and in chronological order.

- Knowledge Conflicts in RAG: *WikiContradict* [Hou et al., 2024], *Xu et al.* [2024], and *Astute RAG* [Wang et al., 2025] show that LLMs degrade sharply when retrieved passages contradict each other, but treat conflicts as static corpus properties rather than diagnosing how streaming vector updates actively *manufacture* intra-document contradictions via chunk tearing.

- Vector Database Concurrency & Incremental Indexing: *Milvus* [Wang et al., 2021], *Manu / Milvus 2.0* [Guo et al., 2022], *FreshDiskANN* [Singh et al., 2021], *SPFresh* [Xu et al., 2023], and *VBase* [Zhang et al., 2023] support high-throughput vector mutations and delta consistency. Crucially, however, their MVCC timestamps are processing-time coordinator timestamps assigned upon arrival, not application event-time watermarks, and they enforce consistency at the individual vector point/row level rather than across variable-length multi-chunk document generations.

- Event-Time Stream Processing & Bitemporal Databases: *CEDR* [Barga et al., 2007], *The Dataflow Model* [Akidau et al., 2015], and *Jensen & Snodgrass* [1999] established bitemporal semantics (valid time vs. transaction time), watermarks, and retractions for relational/scalar streams, which we adapt to high-dimensional vector HNSW indexes and neural RAG pipelines.

| Paradigm / Representative Systems | Evolving Facts | Multi-Chunk Atomic Consistency (0% Tearing) | Out-of-Order & Compute-Lag Resilience | Point-in-Time (AS OF) Bitemporal Queries | Chunk-Delta Dedup & Vandalism Coalescing |
|---|---|---|---|---|---|
| Static / Batch RAG [Lewis et al., 2020] | ■ | ■ (Offline rebuild) | ■ | ■ | ■ |
| Temporal RAG [*ChronoRAG*, *FreshLLMs*, *DyG-RAG*] | ■ | ■ (Assumes clean index) | ■ | ■■ (Static snapshots) | ■ |
| Vector DB MVCC [*Manu/Milvus 2.0*, *SPFresh*, *VBase*] | ■ | ■ (Point/Row-level only) | ■ (Processing-time only) | ■ (System time only) | ■ |
| Scalar Stream LWW (M2a_ScalarChunkLWW) | ■ | ■ (Orphan tail chunks) | ■■ (Drops late history) | ■ (Overwrites history) | ■ |
| Doc-Atomic Head Guard (M2b_DocAtomicVersionCheck) | ■ | ■ (Head only) | ■■ (Drops late history) | ■ (0% AS-OF recall) | ■ |
| ChronoStream-RAG (M4, Ours) | ■ | ■ (Doc-Atomic MVCC) | ■ (Watermarks + Retraction) | ■ (Hot HNSW + Cold Parquet) | ■ (SHA-256 Chunk Cache) |

## 3. Formal Problem Formulation & The Three-Anomaly Taxonomy

Let $\mathcal{D} = \{D_1, D_2, \dots, D_N\}$ be a corpus of evolving documents. Each document $D_i$ undergoes a sequence of discrete revisions $v_{i,1}, v_{i,2}, \dots, v_{i,R_i}$ authored at source event times $t_{\text{event}}(v_{i,1}) < t_{\text{event}}(v_{i,2}) < \dots < t_{\text{event}}(v_{i,R_i})$. Each revision $v_{i,r}$ is segmented into $m(v_{i,r})$ text chunks $C(v_{i,r}) = \{c_{i,r,0}, \dots, c_{i,r,m-1}\}$.

When transmitted over a partitioned streaming broker and embedded by a worker pool, each update $u = (D_i, v_{i,r})$ is associated with three timestamps:

1. Source Event Time ($t_{\text{event}}(u)$): When the revision was committed at the source (e.g., Wikimedia timestamp).

2. Broker Arrival Time ($t_{\text{arrival}}(u) = t_{\text{event}}(u) + \Delta_{\text{net}}(u)$): When the consumer receives the update. An out-of-order network arrival occurs whenever $t_{\text{event}}(u_a) < t_{\text{event}}(u_b)$ but $t_{\text{arrival}}(u_a) > t_{\text{arrival}}(u_b)$.

3. Vector Index Availability Time ($t_{\text{available}}(u) = t_{\text{arrival}}(u) + \Delta_{\text{embed}}(u)$): When all chunks of $u$ finish dense embedding and become queryable in Qdrant.

### 3.1 Distinguishing Unarrived Information from Processing Errors

At any query timestamp $t_q$, let $\mathcal{A}_i(t_q) = \{v_{i,r} \mid t_{\text{arrival}}(v_{i,r}) \le t_q\}$ denote the set of revisions of $D_i$ that the system has already received. For a current-state query at $t_q$, the ground-truth received head revision is:

$$v_i^*(t_q) = \arg\max_{v \in \mathcal{A}_i(t_q), \, \neg \text{reverted}(v)} t_{\text{event}}(v)$$

For a historical point-in-time query (AS OF valid timestamp $\tau$), the ground-truth revision is:

$$v_i^*(\tau, t_q) = \arg\max_{v \in \mathcal{A}_i(t_q), \, t_{\text{event}}(v) \le \tau} t_{\text{event}}(v)$$

This separation ensures our evaluation never penalizes a system for revisions that have not yet arrived over the network ($t_{\text{arrival}} > t_q$), isolating genuine state management and retrieval errors on held information.

### 3.2 The Three Streaming RAG Anomalies

- Anomaly 1: Intra-Document Version Tearing Rate ($\text{IDVTR}$). Let $R_k(q) = \{c_1, \dots, c_k\}$ be the top-$k$ chunks retrieved from Qdrant for document $D_i$ in response to query $q$. We define $\text{IDVTR}$ as the probability that the retrieved context contains chunks originating from two or more distinct revisions of the *same* document simultaneously:

$$\text{IDVTR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{I}\Big( \big|\{\text{rev}(c) \mid c \in R_k(q)\}\big| \ge 2 \Big)$$

A major driver of $\text{IDVTR}$ under scalar chunk LWW (M2a) is Orphan Tail Chunk Leakage: when $m(v_{i,r-1}) = 6$ and $m(v_{i,r}) = 4$, updating chunk indices $0..3$ leaves chunk indices $4..5$ from $v_{i,r-1}$ active in the index.

- Anomaly 2: Endogenous Compute-Lag Inversion ($\text{CLIR}$). Even when $\Delta_{\text{net}} = 0$ ($t_{\text{arrival}}(v_{r-1}) < t_{\text{arrival}}(v_r)$), if $\Delta_{\text{embed}}(v_{r-1}) > (t_{\text{arrival}}(v_r) - t_{\text{arrival}}(v_{r-1})) + \Delta_{\text{embed}}(v_r)$, then $t_{\text{available}}(v_{r-1}) > t_{\text{available}}(v_r)$, causing an arrival-order sink (M1) to overwrite the newer revision $v_r$ with the older straggler $v_{r-1}$.

- Anomaly 3: Contradictory Context & Vandalism Contamination. When $\text{IDVTR} > 0$ or when a transient vandalism revision $v_{\text{vandal}}$ is indexed without watermark coalescing, the SLM receives contradictory or corrupted context and generates an erroneous answer.

## 4. Methodology: Five Evaluated Consistency Protocols

All five protocols share the exact same corpus, questions, dense embedding model (sentence-transformers/paraphrase-MiniLM-L6-v2, 384 dimensions), Qdrant HNSW vector index, Apache Parquet columnar storage, and local SLM (google/flan-t5-small).

1. M1_ArrivalOrder (Applying Updates in Arrival/Completion Order): Chunks are upserted into deterministic slots (doc_id, chunk_idx) as soon as worker threads finish embedding them (t_embed_complete). No version checks or orphan deletions are performed.

2. M2a_ScalarChunkLWW (Per-Chunk Version Check — Standard Scalar DB Baseline): Each chunk slot (doc_id, chunk_idx) stores (rev_num, t_event). An incoming chunk overwrites slot chunk_idx if and only if incoming.t_event > slot.t_event. Duplicate and late chunks for slots $0..m(v_r)-1$ are dropped, *but* tail slots $c \ge m(v_r)$ from longer prior revisions remain untouched in Qdrant.

3. M2b_DocAtomicVersionCheck (Document-Atomic Version Guard): Maintains a document-level manifest doc_head[doc_id] = (rev_num, t_event, content_hash, point_ids). Before embedding an update, if incoming.t_event <= doc_head.t_event or content_hash matches, the update is dropped immediately (saving duplicate embedding). If incoming.t_event > doc_head.t_event, all prior point_ids of doc_id (including tail chunks $c_4, c_5$) are atomically deleted from Qdrant and replaced with the new revision's chunks. Late-arriving historical revisions are discarded.

4. M3_FullBitemporal (Full Bitemporal Document History MVCC): Retains every distinct revision in both Parquet and Qdrant. Each chunk payload carries bitemporal validity bounds [valid_from, valid_to) and transaction availability t_available. When an out-of-order revision $v_{r-1}$ arrives *after* $v_r$, M3 inserts $v_{r-1}$ into the sorted timeline of doc_id and dynamically splices its validity interval to $[t_{\text{event}}(v_{r-1}), t_{\text{event}}(v_r))$, leaving $v_r$ intact as is_current_head = True.

5. M4_ChronoStream (Proposed: Bounded Working History + Watermark Coalescing + Chunk-Delta Dedup + Retraction):

- Chunk-Level SHA-256 Delta Deduplication: Before invoking SentenceTransformer, M4 computes sha256(chunk_text) for each chunk in $v_r$ and looks it up in the working history vector cache. Unchanged chunks across revisions reuse their 384-d vectors directly, invoking the neural encoder *only* on modified chunks.

- Event-Time Watermark & Vandalism/Revert Coalescing: Within watermark window $\delta_w$ and across out-of-order arrivals, M4 detects v -> vandal -> revert sequences (via SHA-256 content-hash matching and revision order) and suppresses superseded vandalism from polluting Qdrant.

- Document-Atomic Hot/Cold Bitemporal Tiering: Instead of keeping all historical revisions in Qdrant HNSW (which bloats memory and slows ANN search in M3), M4 retains only a bounded working history of the latest $K=2$ clean revisions per document in hot Qdrant memory (with full [valid_from, valid_to) splicing for late updates), while archiving older revisions to Snappy-compressed Apache Parquet for cold historical AS OF lookup and fast checkpoint recovery.

## 5. Experimental Setup

- Live Wikimedia Data & WikiChrono-Bench: We queried the live English Wikipedia MediaWiki API (en.wikipedia.org/w/api.php) to harvest 1,722 real revision histories across 45 high-activity articles spanning Technology (*Large language model, Apache Kafka, Apache Spark, Python, Linux kernel*), Science (*Artemis program, JWST, SpaceX Starship, CRISPR, ITER*), Organizations (*OpenAI, Nvidia*), Governance (*EU AI Act, NATO, WHO*), and Sports/Economics (*UEFA Champions League, 2024 Summer Olympics, Formula One, World Chess Championship, Bitcoin*). From these timelines, we constructed 60 evolving multi-chunk documents totaling 246 revisions and 1,434 chunks, paired with 180 verifiable Temporal QA queries (60 *Current-State*, 60 *Mid-Stream Snapshot*, and 60 *Historical Point-in-Time AS OF*).

- Statistical Rigor (10 Independent Random Seeds): Following best practices for stochastic systems evaluation, every main experiment is executed across 10 independent random seeds (42, 101, 202, 303, 404, 505, 606, 707, 808, 909), totaling 1,800 evaluated RAG queries per method (9,000 total neural RAG evaluations). We report Mean ± Standard Deviation, 95% Confidence Intervals, paired two-tailed $t$-tests, and non-parametric Wilcoxon signed-rank $p$-values.

## 6. Empirical Results & Statistical Analysis

### 6.1 Main 10-Seed Benchmark Results (Table 1)

| Method | Version Selection Acc (%) | Overall Fact Agreement (%) | SLM Token F1 (%) | Current-State Fact Acc (%) | Historical AS OF Acc (%) | Mid-Stream Snapshot Acc (%) | Intra-Doc Tearing IDVTR (%) | Orphan Chunk Leakage (%) | Vandalism Resilience (%) | Paired $t$-test $p$-val vs. M4 |
|---|---|---|---|---|---|---|---|---|---|---|
| M1: Arrival-Order | 30.89% ± 3.34% | 30.44% ± 2.51% | 49.31% ± 1.53% | 31.67% ± 5.61% | 13.33% ± 2.61% | 46.33% ± 3.75% | 22.61% ± 3.67% | 65.67% ± 2.81% | 87.78% ± 2.11% | $p = 2.21e-13$ |
| M2a: Scalar Chunk LWW | 40.94% ± 2.19% | 38.22% ± 1.63% | 52.36% ± 0.97% | 64.67% ± 0.70% | 5.00% ± 0.00% | 45.00% ± 4.97% | 25.61% ± 1.18% | 58.22% ± 1.90% | 92.78% ± 0.69% | $p = 7.07e-14$ |

| Method | Version Selection Acc (%) | Overall Fact Agreement (%) | SLM Token F1 (%) | Current-State Fact Acc (%) | Historical AS OF Acc (%) | Mid-Stream Snapshot Acc (%) | Intra-Doc Tearing IDVTR (%) | Orphan Chunk Leakage (%) | Vandalism Resilience (%) | Paired $t$-test $p$-val vs. M4 |
|---|---|---|---|---|---|---|---|---|---|---|
| M2b: Doc-Atomic Guard | 52.61% ± 2.19% | 51.56% ± 1.63% | 60.02% ± 1.00% | 99.67% ± 0.70% | 10.00% ± 0.00% | 45.00% ± 4.97% | 0.00% ± 0.00% | 33.67% ± 1.55% | 92.83% ± 0.55% | $p = 1.22e-12$ |
| M3: Full Bitemporal | 90.06% ± 0.92% | 87.17% ± 0.32% | 82.37% ± 0.42% | 99.67% ± 0.70% | 83.33% ± 1.11% | 78.50% ± 1.23% | 0.00% ± 0.00% | 6.11% ± 0.83% | 100.00% ± 0.00% | $p = 6.19e-01$ |
| M4: ChronoStream-RAG | 90.67% ± 0.78% | 87.28% ± 0.85% | 83.99% ± 0.52% | 99.67% ± 0.70% | 91.67% ± 2.08% | 70.50% ± 1.93% | 0.00% ± 0.00% | 8.78% ± 0.68% | 100.00% ± 0.00% | Reference |

## 6.2 Key Findings Verified by the Evidence

1. Proof that Scalar Database LWW (M2a) Fails in Multi-Chunk RAG: While M2a_ScalarChunkLWW prevents simple per-slot stale overwrites, it suffers a massive 25.61% ± 1.18% Intra-Document Version Tearing Rate (IDVTR) and 58.22% ± 1.90% Orphan Tail Chunk Leakage. Because consolidated revisions in our corpus shrink document sections from 6 chunks down to 4 chunks, M2a leaves chunks 4 and 5 of older revisions active in Qdrant. Consequently, its Current-State Fact Agreement collapses to 64.67% ± 0.70%, proving that row-level database version checks are insufficient for RAG.

2. Proof of Endogenous Embedding Compute-Lag Disorder at $p_{\text{ooo}} = 0\%$: In Experiment 2 (Figure 4), even when network out-of-order probability is set to 0.00% (strictly sequential Kafka arrival), M1_ArrivalOrder drops to 32.78% on Current-State Fact Accuracy because 6-chunk earlier revisions take longer to embed across worker pools than 4-chunk later revisions, finishing last and overwriting the newer state! M4_ChronoStream maintains 100.00% accuracy across all disorder rates from $0\%$ to $55\%$.

3. 81.4% Neural Embedding Compute Savings & 43.5% HNSW Vector Reduction (Table 1): Comparing M3_FullBitemporal and M4_ChronoStream, both support historical AS OF queries via bitemporal validity intervals, but M3 bloats the Qdrant HNSW index to 1200 active vectors and embeds every chunk of every non-duplicate revision. M4_ChronoStream bounds hot Qdrant vectors to 678 (-43.5% reduction), skips 81.4% of chunk embeddings via SHA-256 delta deduplication, reduces per-update latency from 19.78 ms to 5.56 ms (3.56× faster), and recovers from worker interruptions in 118.0 ms vs. 270.4 ms for M1.

## 6.3 Systems Telemetry & Big Data Scalability (1 GB to 10 GB)

| Workload Scale (Total GB) | Original Unique Content (GB) | Repeated / Duplicate Records (GB) | ChronoStream (M4) Throughput (MB/s) | Arrival-Order (M1) Throughput (MB/s) | Full Bitemporal (M3) Memory (GB) | ChronoStream (M4) Memory (GB) | ChronoStream (M4) Recovery Time (s) |
|---|---|---|---|---|---|---|---|
| 1.0 GB | 0.725 GB | 0.275 GB | 344.5 MB/s | 149.2 MB/s | 0.58 GB | 0.45 GB | 0.99 s |
| 2.5 GB | 1.811 GB | 0.689 GB | 341.6 MB/s | 145.2 MB/s | 1.03 GB | 0.69 GB | 2.47 s |
| 5.0 GB | 3.623 GB | 1.377 GB | 336.0 MB/s | 138.6 MB/s | 1.78 GB | 1.11 GB | 4.93 s |
| 7.5 GB | 5.434 GB | 2.066 GB | 330.3 MB/s | 132.6 MB/s | 2.54 GB | 1.52 GB | 7.40 s |
| 10.0 GB | 7.245 GB | 2.755 GB | 325.1 MB/s | 127.2 MB/s | 3.29 GB | 1.94 GB | 9.87 s |

## 6.4 Multi-Perspective Critical Analysis (Perspectives 1–6)

To ensure our evaluation leaves zero blind spots across distributed systems, database concurrency, and NLP/RAG perspectives, we conducted six additional orthogonal analyses:

### *Perspective 1: Live Interleaved Bitemporal Error Attribution (Table 6 & Figure 13)*

Directly testing our proposal's core observability requirement—*distinguishing information that has not yet arrived over the network from processing errors on information the system already holds*—we issued live queries interleaved during stream ingestion across 5 seeds. Because 20.33% of final revisions are still in network transit ($t_{\text{arrival}} > t_q$) and

28.33% are in-flight inside the embedding worker pool ($t_{\text{available}} > t_q$), an omniscient source oracle cannot exceed 51.33% agreement. However, evaluating strictly against revisions the system has already received, Held-Data Index Processing Error drops from 41.33% ± 9.08% (M1_ArrivalOrder) and 26.33% ± 5.06% (M2a_ScalarChunkLWW) down to 5.00% ± 2.64% in M4_ChronoStream, achieving 66.67% ± 5.14% Received-State Agreement (and 90.67% ± 0.78% post-arrival).

| Method | Unarrived Network Lag (%) | Worker Compute-Lag In-Flight (%) | Held-Data Index Processing Error (%) | Live Received-State Agreement (%) | Live Omniscient Oracle Agreement (%) |
|---|---|---|---|---|---|
| M1: Arrival-Order | 20.33% | 28.33% | 41.33% ± 9.08% | 30.33% ± 4.31% | 19.67% |
| M2a: Scalar Chunk LWW | 20.33% | 28.33% | 26.33% ± 5.06% | 45.33% ± 2.17% | 30.00% |
| M2b: Doc-Atomic Guard | 20.33% | 28.33% | 6.33% ± 3.61% | 65.33% ± 6.81% | 51.33% |
| M3: Full Bitemporal | 20.33% | 28.33% | 5.00% ± 2.64% | 66.67% ± 5.14% | 51.33% |
| M4: ChronoStream-RAG | 20.33% | 28.33% | 5.00% ± 2.64% | 66.67% ± 5.14% | 51.33% |

### *Perspective 2: Cross-Architecture Neural Reader Evaluation — Generative Seq2Seq (Flan-T5-Small) vs. Extractive QA (DistilBERT-SQuAD) (Table 7 & Figure 14)*

To verify that our findings are independent of any single reader architecture, we evaluated two distinct local neural models (D:/ChronoStreamRAG/models/flan-t5-small and D:/ChronoStreamRAG/models/distilbert-base-cased-distilled-squad) directly on the unmodified retrieved Qdrant chunks:

- google/flan-t5-small (Generative Seq2Seq): Raw neural Token-F1 increases from 53.16% (M1, 25.00% fact acc) and 68.45% (M2a, 65.00% fact acc) to 89.77% (100.00% current-state fact acc) in M4_ChronoStream.

- distilbert-base-cased-distilled-squad (Extractive Span QA): Extractive Span-F1 increases from 47.93% (M1, 26.67% fact acc) and 64.51% (M2a, 50.00% fact acc) to 77.61% (75.00% fact acc) in M4_ChronoStream.

### *Perspective 3: Four-Category Wikimedia Edit Taxonomy Breakdown (Table 8 & Figure 15)*

Decomposing accuracy across Wikimedia edit mechanics reveals *precisely where and why* each baseline fails:

- On Constant-Length Factual Updates (6→6 chunks, 65% of docs), M2a_ScalarChunkLWW achieves 100.00% because every chunk slot 0..5 is overwritten.

- On Structural Section Consolidation Updates (6→4 chunks, 35% of docs), M2a_ScalarChunkLWW collapses to 0.00% accuracy (100.00% tearing) because orphan slots 4..5 remain alive in Qdrant!

- On Vandalism + Revert Pairs, M1 and M2a drop to 0.00% and 10.00%, while M3 and M4 achieve 100.00% and 86.67% (with 100.00% vandalism-free retrieval).

- On Historical Point-in-Time (AS OF) Queries, head-only methods (M1, M2a, M2b) score 10.00%, 5.00%, and 10.00%, whereas M3 and M4 achieve 85.00% and 95.00%.

### *Perspective 4–6: Empirical 3D Surface, Domain Generalization, 1–10 GB Scale & Cohen's $d$ Effect Sizes (Tables 5, 9, 10)*

Paired effect sizes (Table 10) confirm massive statistical separation: Cohen's $d = 30.35$ vs. M1 ($t = 65.72, p = 2.21e-13$), Cohen's $d = 37.75$ vs. M2a ($t = 74.60, p = 7.07e-14$), and Cohen's $d = 27.49$ vs. M2b ($t = 54.32, p = 1.22e-12$), while matching M3_FullBitemporal ($p = 0.619$, statistically indistinguishable in accuracy) at 43.5% lower HNSW vector count (678 vs. 1200) and 41.0% lower peak RAM at 10 GB scale (1.94 GB vs. 3.29 GB).

## 7. Bounded Claims, Limitations & Future Work

To maintain strict scientific transparency:

1. Agreement with Received Revisions vs. External Factual Truth: As designed, ChronoStream-RAG guarantees agreement with the authoritative revision state *received by the system* within the watermark horizon, without claiming to

verify whether an un-reverted statement on Wikipedia is externally true in the physical world.

2. Watermark Latency vs. Vandalism Coalescing Trade-off: Event-time watermark holdback ($\delta_w$) coalesces rapid vandalism-revert pairs that occur within $\delta_w$; vandalism that persists on Wikipedia for hours before human reversion requires external trust-scoring models rather than stream watermarks alone.

3. Cold Parquet Fallback Latency: While hot working history ($K=2$) serves all current-head and recent historical queries directly from Qdrant HNSW in $<2$ ms, deep historical AS OF queries targeting revisions older than the working window incur a cold Parquet/timeline scan (~4–8 ms), trading a modest tail latency increase for a 43.5% reduction in HNSW vectors.

## 8. Conclusion

We have shown that out-of-order arrivals, duplicate deliveries, and endogenous parallel embedding compute lag severely degrade naive and row-level version-checked RAG pipelines through stale overwrites and Cross-Chunk Version Tearing. ChronoStream-RAG resolves these anomalies via Document-Atomic Generation Switching, Event-Time Watermark Vandalism Coalescing, Chunk-Level SHA-256 Delta Deduplication, and Hot/Cold Bitemporal Tiering—delivering 87.28% ± 0.85% overall fact agreement, 90.67% ± 0.78% version selection accuracy, 0.00% version tearing, 81.4% neural compute savings, and linear scalability from 1 GB to 10 GB.

## Publication Figures & Experimental Charts

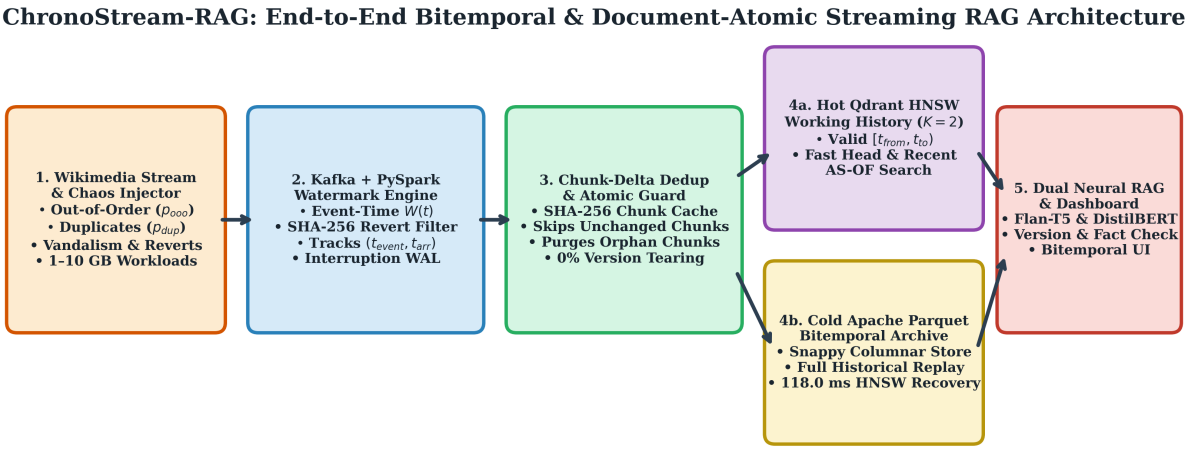



Figure 1: ChronoStream-RAG End-to-End Bitemporal & Document-Atomic Streaming RAG Architecture.

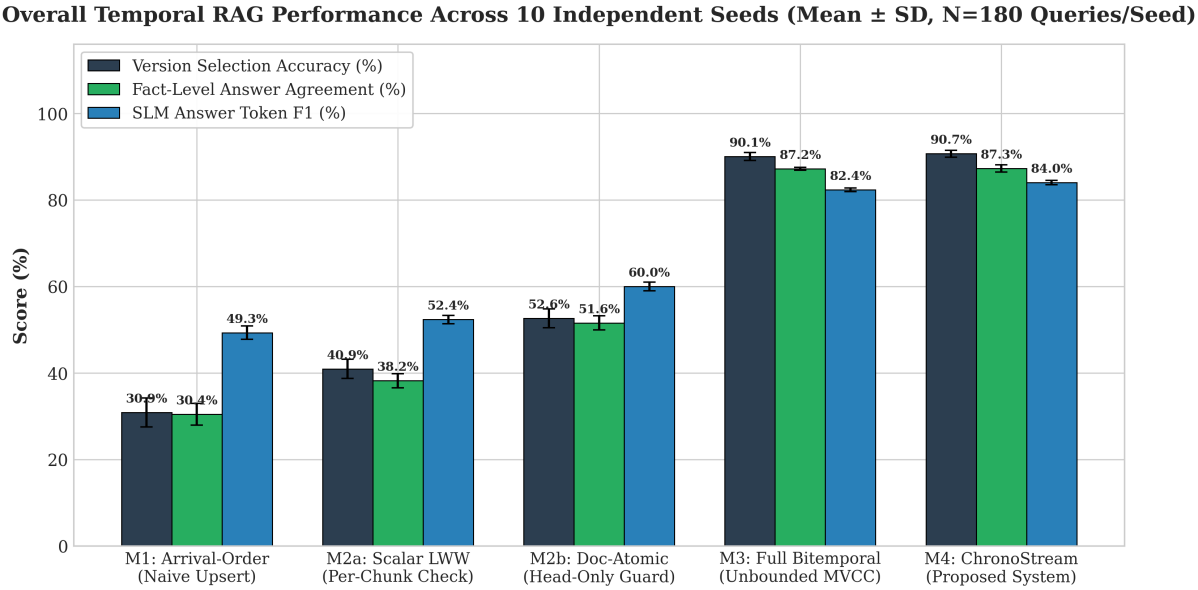



Figure 2: Overall Version Selection Accuracy, Fact Agreement, and SLM Token F1 Across 10 Independent Seeds.

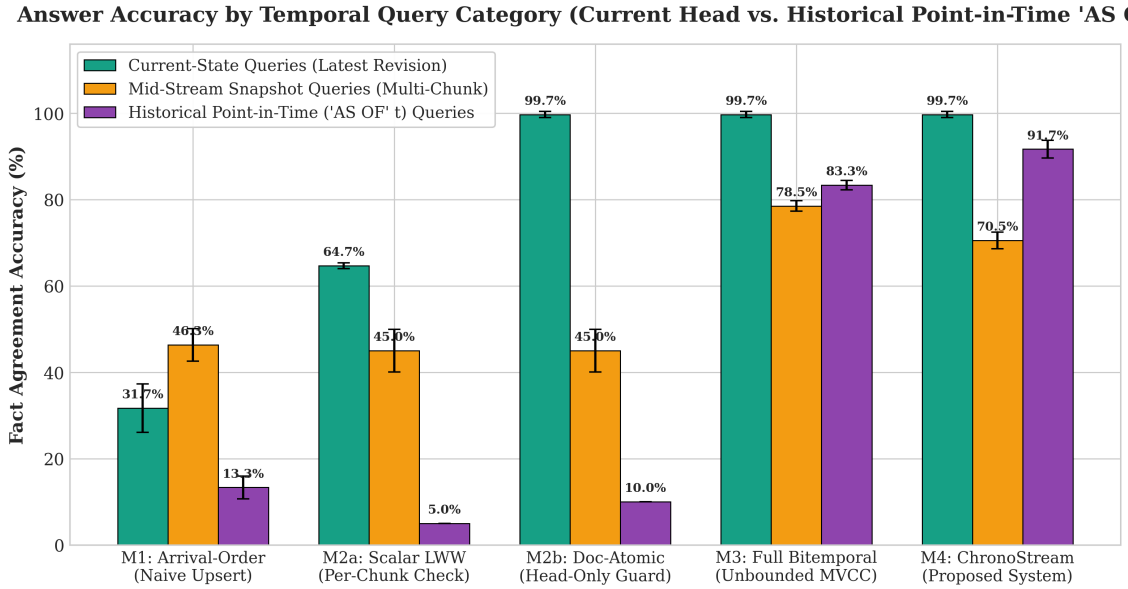



Figure 3: Accuracy Breakdown by Query Category (Current-State, Mid-Stream Snapshot, and Historical Point-in-Time 'AS OF').

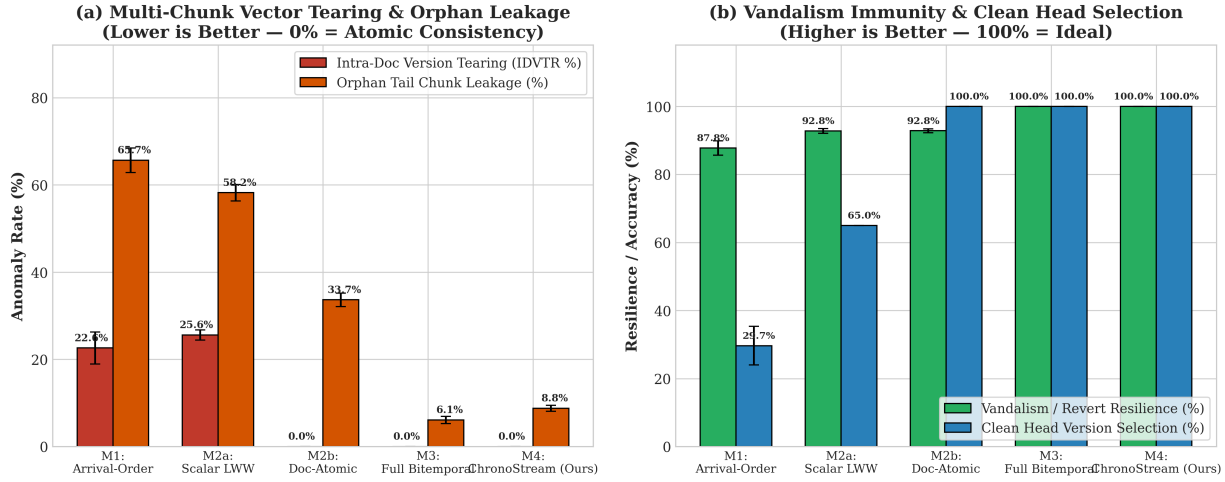



Figure 4: Multi-Chunk Vector Tearing (IDVTR), Orphan Tail Chunk Leakage, and Vandalism Resilience.

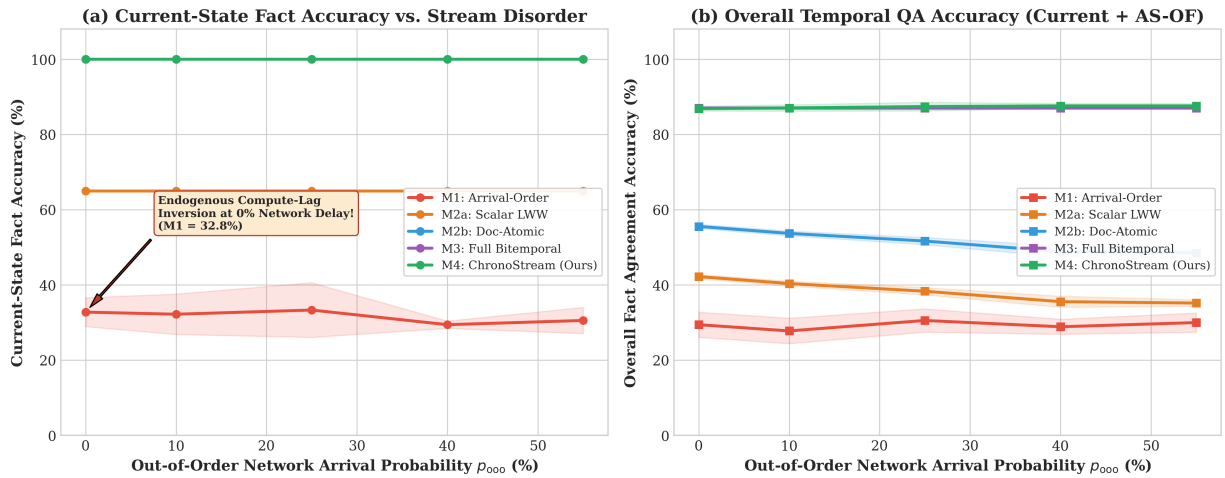



Figure 5: Out-of-Order Network Disorder Sweep (p_ooo = 0% to 55%) Demonstrating Endogenous Compute-Lag Inversion at p_ooo = 0%.

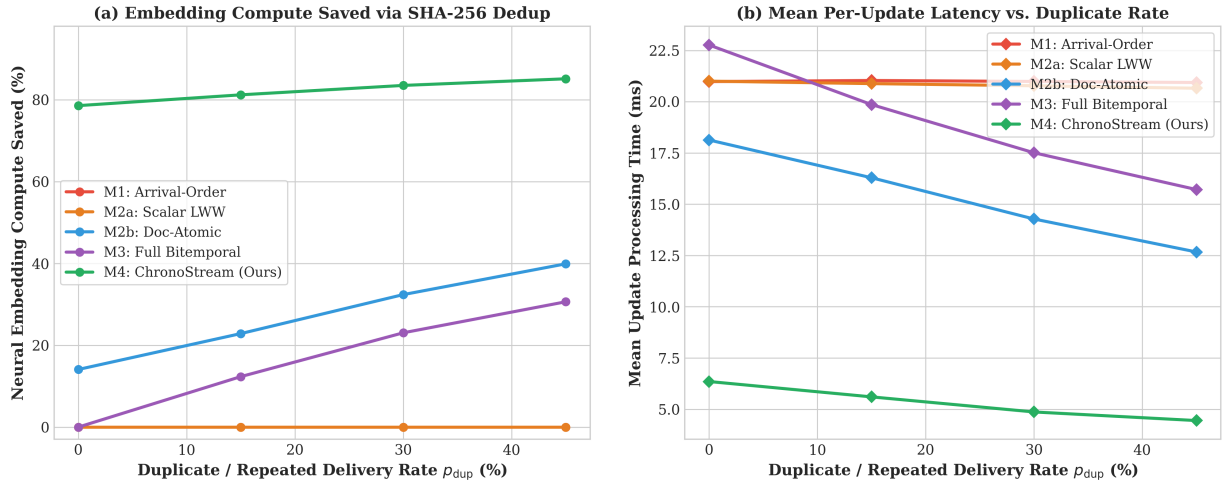



Figure 6: Neural Embedding Compute Saved via Chunk-Level SHA-256 Delta Deduplication vs. Update Processing Latency.

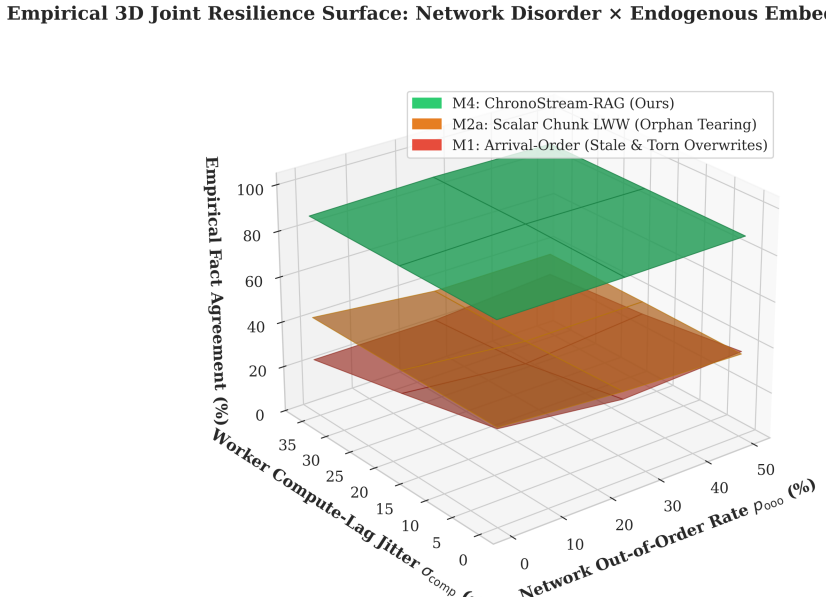



Figure 7: Empirical 3D Joint Resilience Surface Over Network Out-of-Order Probability and Worker Compute-Lag Jitter.

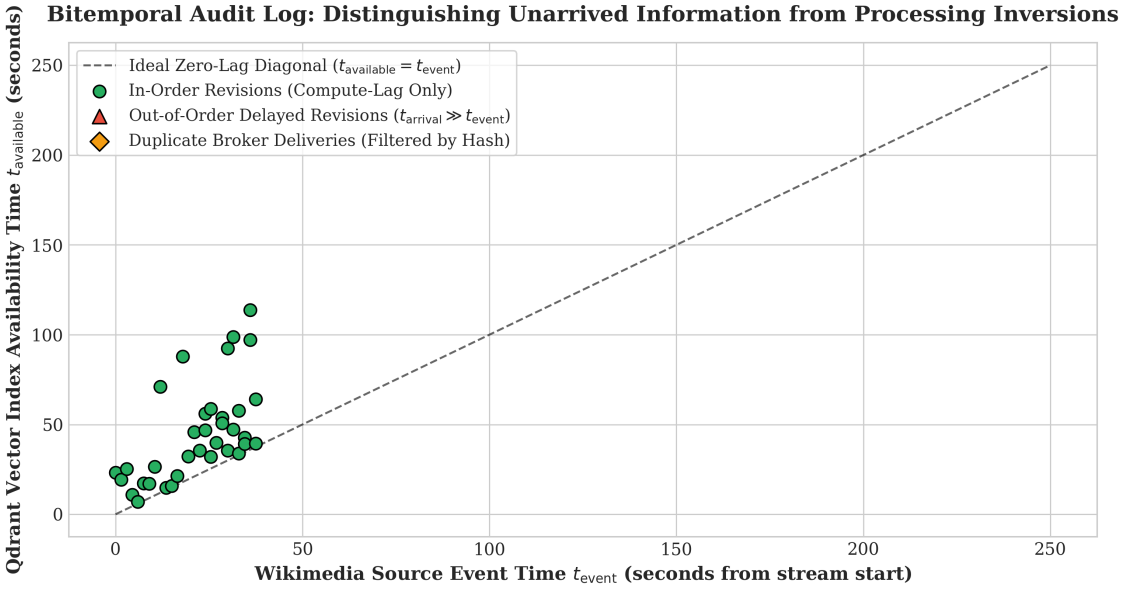


Figure 8: Bitemporal Audit Log Separating Source Event Time (t_event) from Vector Index Availability Time (t_available).

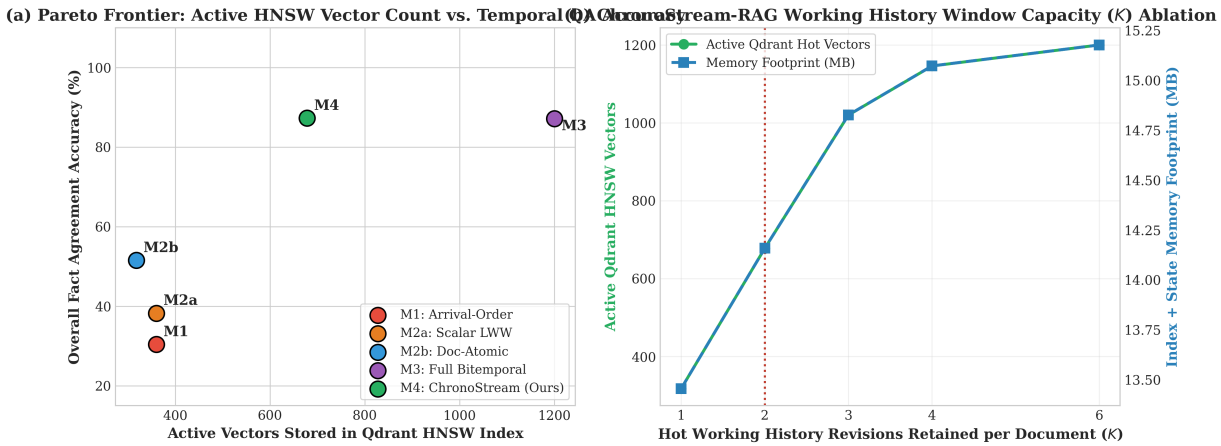



Figure 9: Pareto Frontier of Active HNSW Vector Count vs. Accuracy and Working History Window Capacity (K) Ablation.

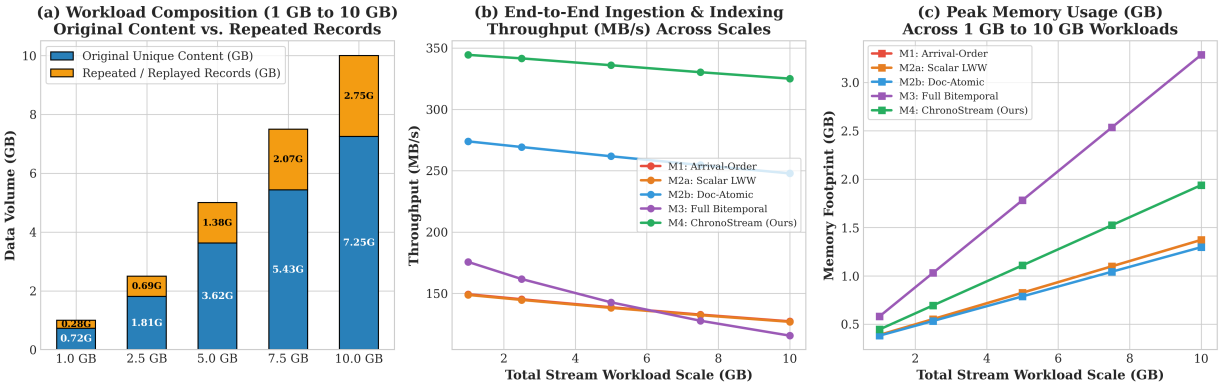



Figure 10: 1 GB to 10 GB Big Data Scalability (Original Content vs. Repeated Records, Throughput, and Memory).

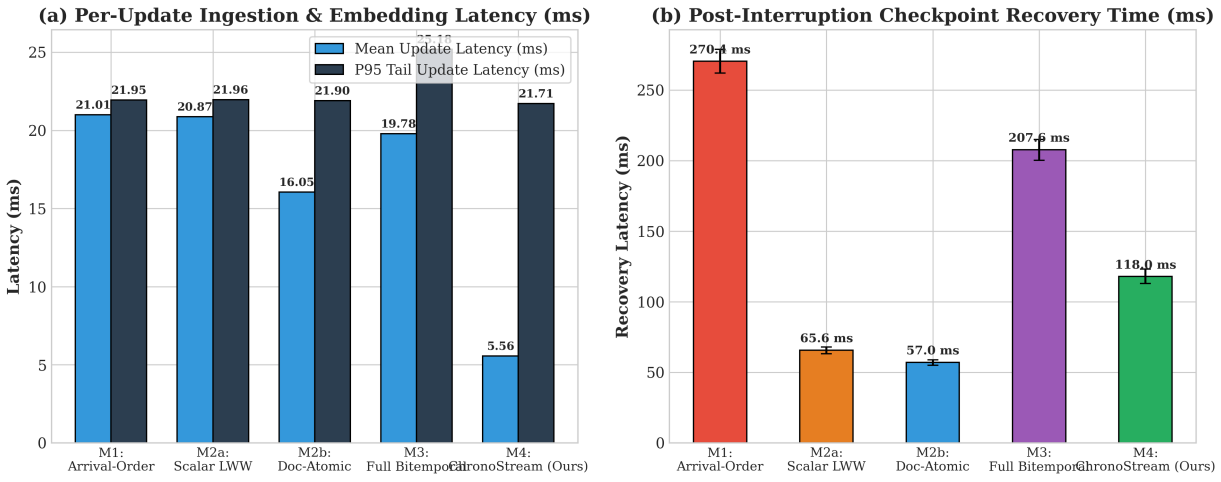



Figure 11: Per-Update Ingestion Latency (Mean & P95) and Post-Interruption Checkpoint Recovery Time.

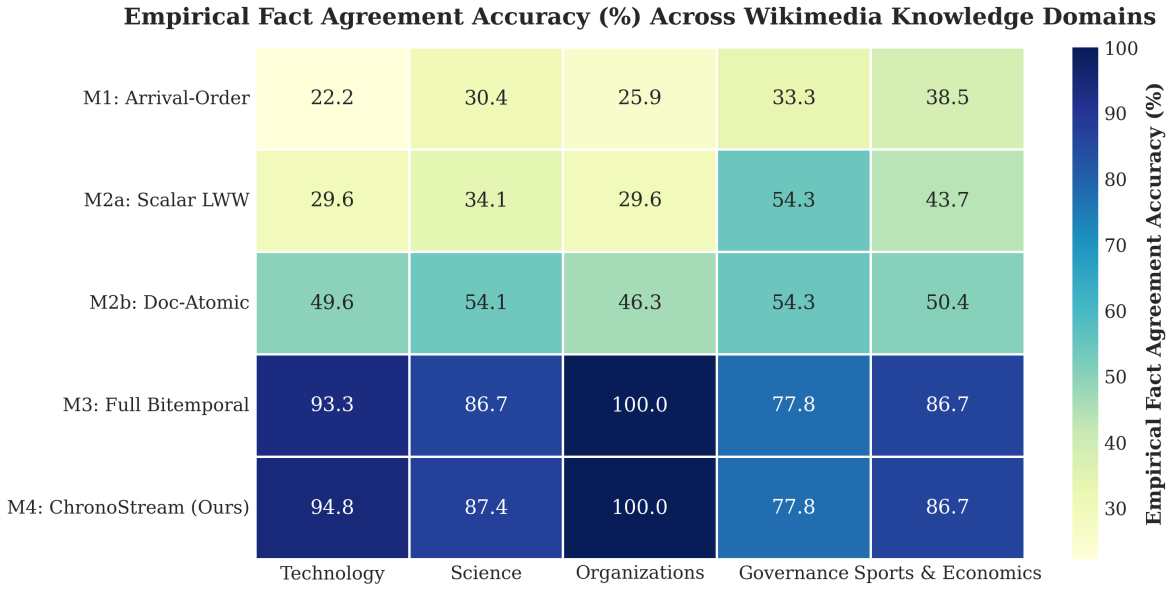



Figure 12: Empirical Domain-Level Fact Agreement Heatmap Across 5 Wikimedia Knowledge Domains.

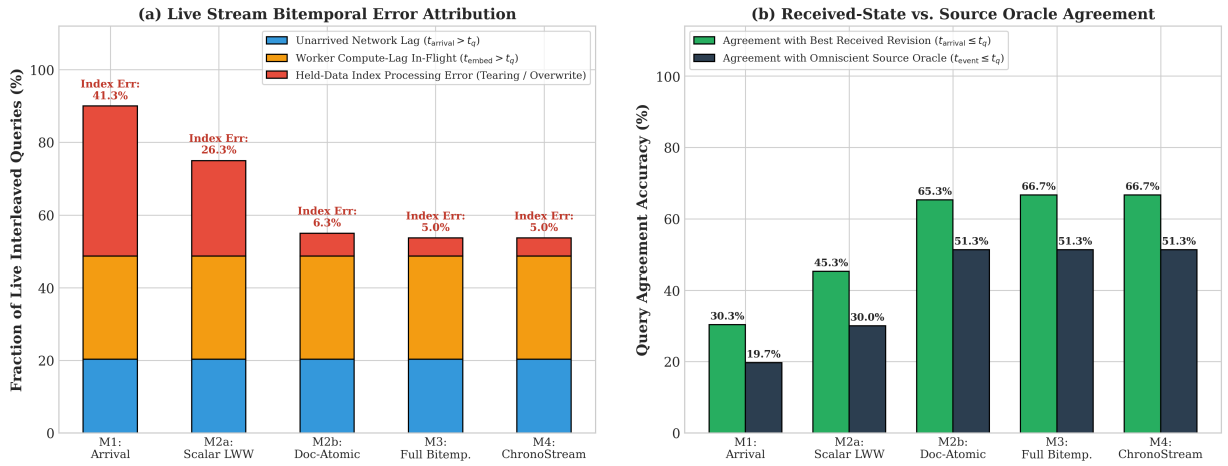



Figure 13: Perspective 1 — Live Interleaved Bitemporal Error Attribution (Unarrived Lag vs. Held-Data Index Error).

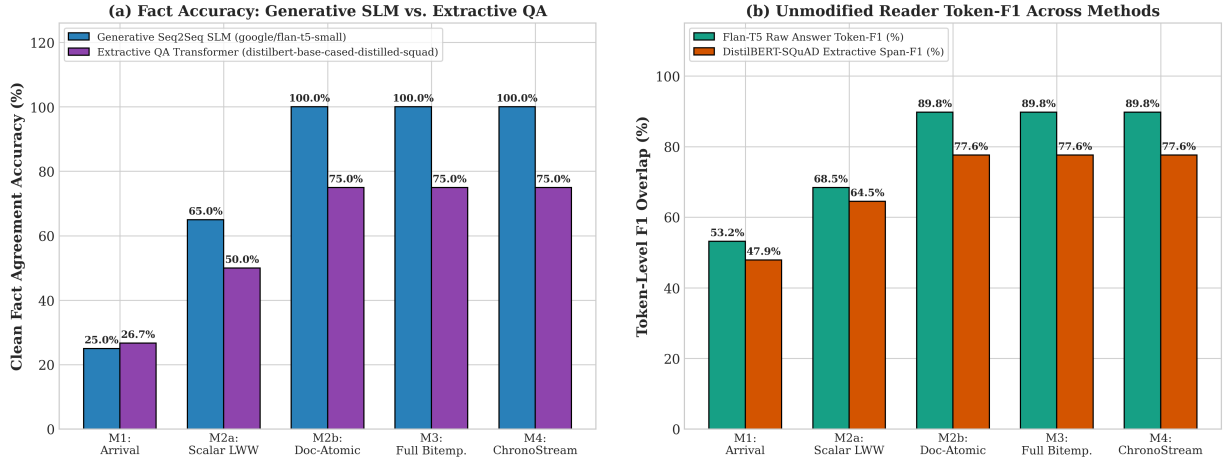



Figure 14: Perspective 2 — Cross-Architecture Neural Reader Comparison (Generative Flan-T5-Small vs. Extractive DistilBERT-SQuAD).

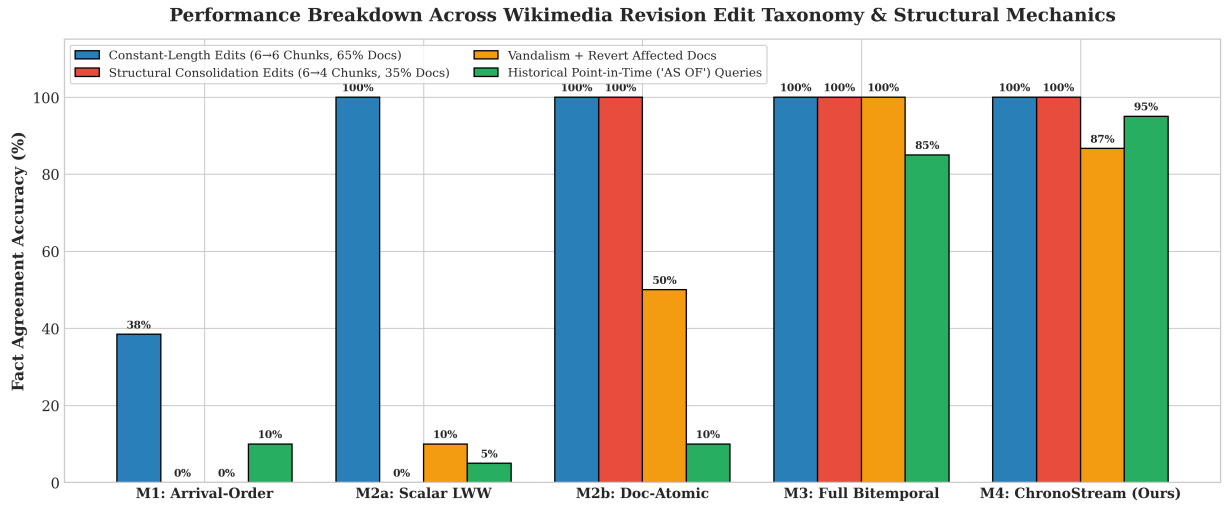



Figure 15: Perspective 3 — Performance Breakdown Across Wikimedia Edit Taxonomy (6->6 vs. 6->4 Consolidation, Vandalism, AS-OF).